

Bakoda — Hotel Room Booking API

Bulut Mimarilerinde Test Mühendisliği — Dönem Projesi Final Raporu

MTH2526-B25 · Marmara Üniversitesi · 2025–2026 Bahar · Konu #15

Grup üyeleri: Arif Batuhan Bahar · Burak Tuzcu

Repo: github.com/BurakTz/bakoda · Haziran 2026

1. Giriş

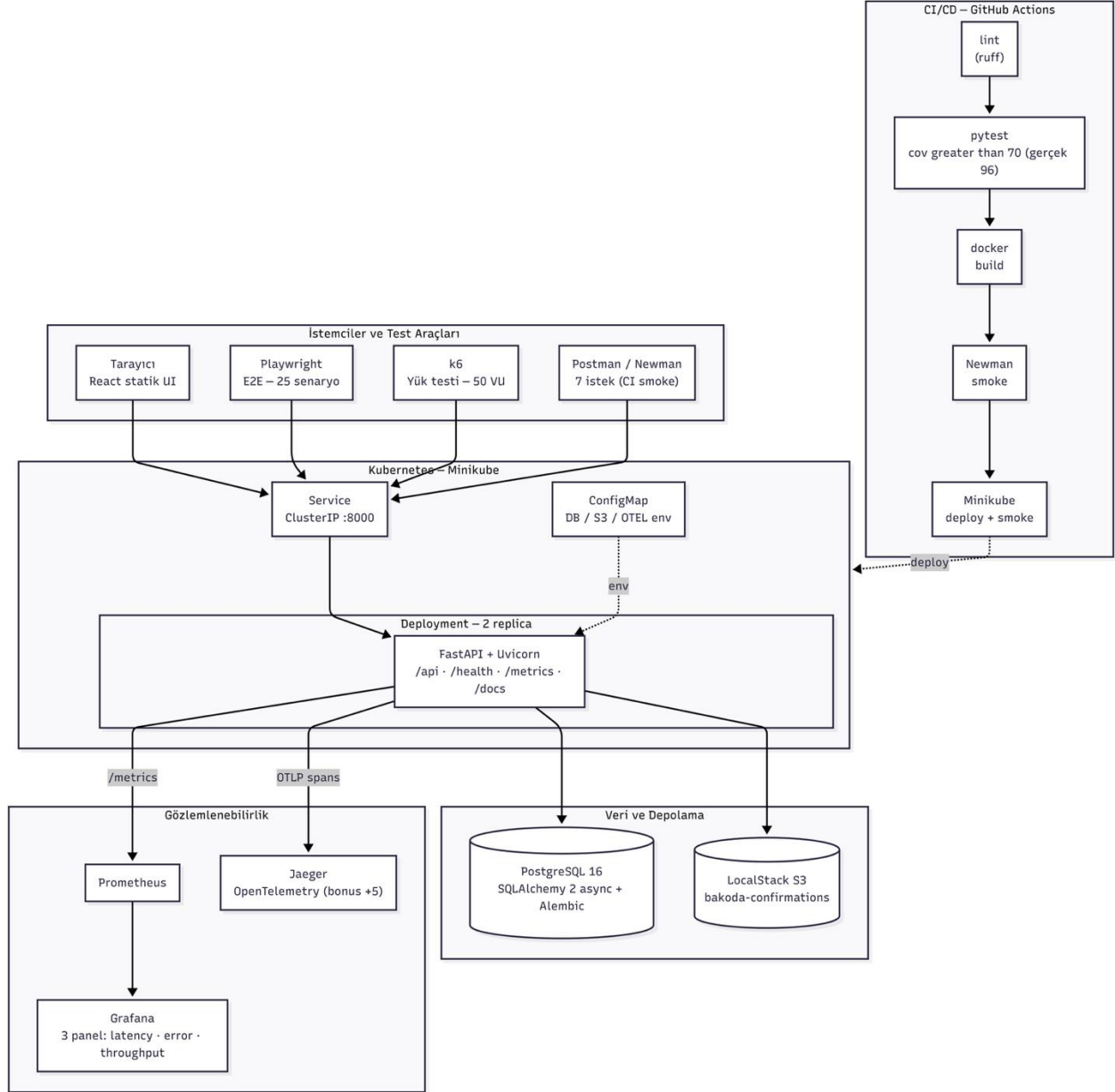
Bu projenin amacı karmaşık bir uygulama yazmak değil, basit bir mini-servis için endüstri standardında test ve dağıtım altyapısı kurmaktır. Şartname kapsamındaki tüm katmanlar — unit/integration/E2E testleri, çok aşamalı container imajı, LocalStack üzerinden AWS emülasyonu, Kubernetes manifestleri, Prometheus/Grafana ile gözlemlenebilirlik, k6 ile performans testi ve GitHub Actions ile CI/CD boru hattı — tek bir repoda birleştirilmiştir.

Seçilen mini-uygulama Bakoda, otel oda rezervasyonu domain'inde (Konu #15) çalışan bir FastAPI mikroservisidir. Kullanıcılar otel/oda arar, tarih aralığına göre müsaitlik kontrolü yapar, rezervasyon oluşturur/günceller/iptal eder; kimlik doğrulama, favoriler, ödeme ve iletişim uçları da mevcuttur. Rezervasyon onayı LocalStack S3'e JSON olarak yazılır.

Konu #15'in seçilme gerekçesi: gerçek hayatta çakışma kontrolü (aynı oda/tarih), durum makinesi (confirmed/cancelled), harici depolama (S3) ve ölçülebilir gecikme gerektiren tipik bir backend senaryosu olması; bu da test piramidi ve gözlemlenebilirlik araçlarını anlamlı biçimde uygulamaya elverişlidir. Proje 2 kişilik grup olarak geliştirilmiştir (bkz. Bölüm 7).

2. Mimari

Sistemin tüm bileşenleri tek sayfalık mimari diyagramda gösterilmiştir (docs/architecture.png; kaynak: docs/architecture.mmd). Ana veri akışı:



Bileşenler ve rolleri:

Bileşen	Teknoloji	Rol
API	FastAPI 0.115 + Uvicorn	REST API; /api, /health, /metrics, OpenAPI /docs (34 uç)
Veritabanı	PostgreSQL 16, SQLAlchemy 2 async, Alembic (5 migration)	User, Hotel, Room, Booking, Favorite, SavedCard, Payment, ContactMessage
Object storage	LocalStack S3 + boto3	Rezervasyon onay belgeleri (bucket: bakoda-confirmations)
Monitoring	prometheus-fastapi-instrumentator	HTTP latency, durum kodu, throughput metrikleri
Dashboard	Grafana + Prometheus	3 panel: p50/p95/p99 latency, 5xx error rate, req/s
Tracing (bonus)	OpenTelemetry OTLP → Jaeger	FastAPI + SQLAlchemy span'leri (config ile açılıp kapanır)
Container	Çok aşamalı Dockerfile	builder (gcc+pip) → runtime (non-root appuser)
Orkestrasyon	Kubernetes (Minikube)	Deployment (2 replica) + Service + ConfigMap

İş mantığı özeti:

- Müsaitlik: booking_service, onaylı rezervasyonlarla tarih aralığı çakışmasını ve otel oda stoğunu kontrol eder; uygun değilse HTTP 409 döner.
- Rezervasyon: oluşturma sırasında DB'ye yazım + S3'e onay JSON'u; PATCH ile tarih/misafir güncelleme (müsaitlik yeniden kontrol + fiyat yeniden hesap); PATCH ../cancel ile iptal.
- Ödeme & iletişim: Payment ve ContactMessage modelleriyle kalıcı kayıt; başarılı ödeme ilgili rezervasyonu onaylı işaretler.

3. Test Stratejisi

Test piramidi tabanda çok sayıda hızlı unit testten, ortada servisleri gerçek veritabanıyla doğrulayan integration testlerine, zirvede ise uçtan uca senaryolara doğru kurulmuştur. Katman dağılımı:

Katman	Test sayısı	Araç / kapsam
Unit	136	pytest + mock; servisler, yardımcıları, şema ve factory doğrulamaları
Integration	145	pytest-asyncio + httpx AsyncClient + Testcontainers PostgreSQL
E2E	25	Playwright (3 dosya): API akışı + UI kullanıcı senaryoları (CI'da koşar)
API smoke	7 istek	Postman koleksiyonu + Newman (CI'da koşar)
Performans	1 senaryo	k6 (health → rooms → book → cancel) — CI'da regresyon kapısı

Coverage: Hedef pyproject.toml'da fail_under = 70'tir; ölçülen gerçek değer %98.5'tir (98.54%). Unit + integration koşumu 293 test geçer, 0 hata. Toplam 306 test fonksiyonu (136 unit + 145 integration + 25 E2E). Çoğu modül %100; en yüksek kapsam servis katmanındadır.

- Test verisi: tests/factories.py içinde Factory Boy + Faker ile RoomFactory, BookingFactory, HotelFactory ve UserFactory; gerçekçi veri üretir ve paylaşılan fixture'lar (sample_hotel/sample_room/book_room) dahil suite genelinde aktif kullanılır.
- Integration altyapısı: confest.py içinde PostgresContainer('postgres:16-alpine'); CI'da TEST_DATABASE_URL ile servis container'a da bağlanabilir. Şema her oturumda Base.metadata ile kurulur.
- Test hijyeni & dayanıklılık: OpenTelemetry test altında kapatılır (otel_enabled bayrağı); senkron SQLAlchemy çağrıları MagicMock ile mock'lanır; LocalStack S3 testleri Docker yoksa hata yerine zarifçe skip edilir.
- Öne çıkan senaryolar: 409 çift-rezervasyon çakışması, iptal sonrası müsaitliğin geri gelmesi, güncellemede oda müsait değilse 409, sahiplik (ownership) doğrulaması, ödeme/iletişim kayıtlarının kalıcılığı.

4. Pipeline ve Deploy

Docker: Çok aşamalı Dockerfile; builder aşamasında bağımlılıklar derlenir, runtime aşamasında yalnızca gerekli paketler non-root appuser ile çalışır ve HEALTHCHECK /health uçuşu yoklar. docker-compose.yml lokal geliştirme için app, postgres, localstack, prometheus, grafana ve jaeger servislerini ayağa kaldırır.

Kubernetes (Minikube) manifestleri:

Manifest	İçerik
k8s/deployment.yaml	2 replica, resource limit, liveness/readiness probe (/health), Prometheus scrape anotasyonları
k8s/service.yaml	ClusterIP, port 8000
k8s/configmap.yaml	DB host, S3 endpoint, bucket, OTEL endpoint

CI/CD (GitHub Actions — .github/workflows/ci.yml):

Boru hattı şartnamedeki sırayı uygular ve genişletir: lint → test → build → smoke → Newman → E2E → k6 → deploy → smoke.

1. lint: ruff check (E, F, I kuralları) — tüm repo temiz.
2. test: pytest --cov=src --cov-fail-under=70 (postgres:16 servis container'ı ile); 293 test, %98.5 coverage.
3. docker build + compose up + /health smoke + Newman (Postman) + Playwright E2E (25 senaryo) + k6 performans kapısı — hepsi canlı stack üzerinde.
4. deploy + smoke: medyagh/setup-minikube ile cluster; image yüklenir, k8s/ manifestleri kubectl apply ile uygulanır, kubectl rollout status beklenir ve servise /health smoke testi yapılır.

Deploy stratejisi: rolling update (Deployment varsayılanı), iki replica ile kesintisiz; readiness probe geçmeden trafik almaz. Rollback için kubectl rollout undo kullanılabilir.

5. Performans ve Gözlemlenebilirlik

k6 yük testi (perf/load-test.js): yerelde 30s→10 VU, 1m→50 VU, 30s→0; CI profilinde (K6_PROFILE=ci) ~35 sn'lik kısa koşu. Her VU sırasıyla health → müsait oda listele → rezervasyon oluştur → iptal akışını koşar. Eşik (her iki profilde): p95 < 500 ms ve hata oranı < %5; ihlalde k6 non-zero exit eder ve CI kırmızı olur — yani gerçek bir regresyon kapısıdır.

Metrik	Değer	Eşik / durum
p95 latency	465.56 ms	< 500 ms ✓
Ortalama latency	69.94 ms	—
Hata oranı	0.00%	< 5% ✓
Toplam HTTP isteği	4756 (≈39.5/s)	—

Yorum: ortalama gecikme düşüktür (~70 ms); p95'in eşiğe yakın olması yoğun yükte POST /api/bookings (DB yazma + S3 yükleme) ve eşzamanlı VU'lardan kaynaklanan kuyruk gecikmesini gösterir. Grafana dashboard'unda (monitoring/grafana-dashboard.json) üç panel bulunur: p50/p95/p99 latency, 5xx error rate ve req/s throughput. Bonus olarak OpenTelemetry ile FastAPI ve SQLAlchemy span'leri Jaeger'a gönderilir.

6. Sonuç ve Öğrenilenler

Sayısal özet:

Metrik	Değer
Domain / konu	Hotel Room Booking (#15)
REST uç sayısı	34 (/api)
Otomatik test (unit+integration)	293 geçer, 0 hata
Toplam test fonksiyonu	306 (136 unit · 145 integration · 25 E2E)
Coverage (gerçek)	%98.5 (hedef ≥%70)
Postman / Newman	7 istek (CI'da)
k6 performans kapısı	p95 < 500 ms (CI regresyon kapısı)
Grafana panel	3
Kubernetes	Deployment (2 replica) + Service + ConfigMap
Bonus	OpenTelemetry + Jaeger (+5)

Karşılaşılan zorluklar:

5. Async SQLAlchemy + Testcontainers: session yaşam döngüsü ve NullPool ile test izolasyonu; integration fixture'ların doğru sıralanması.
6. Birim testlerde async/senkron mock ayrımı: AsyncMock'ın senkron .add() çağrısında coroutine sızdırması, hedefli MagicMock ile giderildi.
7. CI'da gerçek Minikube deploy ve E2E/k6 otomasyonu: app açılışta Alembic migration koştugu için PostgreSQL bağımlılığının cluster içinde de ayağa kaldırılması gerekti; E2E için canlı stack + seed kuruldu.

İleride yapılabilecekler:

- Helm chart paketleme veya ArgoCD ile GitOps (bonus).
- k6 sonuçlarının CI artefaktı olarak otomatik perf/report.md'ye yazılması.
- KEDA ile event-driven autoscaling.

7. İş Paylaşımı

Proje 2 kişilik grup olarak geliştirilmiştir. Modül sorumlulukları:

Modül	Sorumlu	Yardımcı
Frontend (React statik sayfalar)	Arif Batuhan Bahar	Burak Tuzcu
Unit / Integration / E2E testler, test altyapısı	Arif Batuhan Bahar	Burak Tuzcu
GitHub Actions CI (E2E + k6 dahil)	Arif Batuhan Bahar	Burak Tuzcu
Backend (FastAPI route + servisler)	Burak Tuzcu	Arif Batuhan Bahar
DB modelleri, şema, Alembic, seed	Burak Tuzcu	Arif Batuhan Bahar
Docker, Kubernetes, LocalStack S3	Burak Tuzcu	Arif Batuhan Bahar
Monitoring, k6, OpenTelemetry	Burak Tuzcu	Arif Batuhan Bahar
Final rapor & sunum	Arif + Burak	—

Sunum sorumluluğu (20 dk slot): 0–7 dk problem + mimari + test stratejisi; 7–14 dk canlı demo (PR → CI → deploy → metrik → E2E); 14–17 dk sayılar + öğrenilenler; 17–20 dk Q&A — hep birlikte. Detay: docs/work-distribution.md.

8. Kaynaklar

- FastAPI — <https://fastapi.tiangolo.com/>
- pytest & pytest-asyncio — <https://docs.pytest.org/>
- Testcontainers Python — <https://testcontainers-python.readthedocs.io/>
- Factory Boy — <https://factoryboy.readthedocs.io/>
- LocalStack S3 — <https://docs.localstack.cloud/user-guide/aws/s3/>
- prometheus-fastapi-instrumentator — <https://github.com/trallnag/prometheus-fastapi-instrumentator>
- OpenTelemetry Python — <https://opentelemetry.io/docs/languages/python/>
- k6 — <https://grafana.com/docs/k6/latest/>
- Playwright Python — <https://playwright.dev/python/>
- Newman — <https://learning.postman.com/docs/collections/using-newman-cli/>
- Kubernetes (Deployment, Service, ConfigMap) — <https://kubernetes.io/docs/>
- Marmara MTH2526-B25 Dönem Projesi Şartnamesi (Bölüm 3–9)